

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

boston_data =
pd.read_csv("C:/Users/Admin/Downloads/archive/BostonHousing.csv")

boston_data
boston_data.rename(columns={'medv':'Price'}, inplace=True)

boston_data
boston_data.shape
boston_data.head()
boston_data.info()
boston_data.describe()
boston_data.isnull().sum()
type(boston_data)
data = pd.DataFrame()
data.isnull().sum()

import seaborn as sns
sns.distplot(boston_data.Price)

sns.boxplot(boston_data.Price)

correlation = boston_data.corr()
correlation.loc['Price']

import matplotlib.pyplot as plt
fig, axes = plt.subplots(figsize=(15,12))
sns.heatmap(correlation, square=True, annot=True)

plt.figure(figsize=(20,5))
features = ['lstat', 'rm', 'ptratio']
for i, col in enumerate(features):
    plt.subplot(1, len(features), i+1)
    x = boston_data[col]
    y = boston_data['Price']
    plt.scatter(x, y, marker='o')
    plt.title("Variation in House prices")
    plt.xlabel(col)
    plt.ylabel('"House prices in $1000"')
```

```

X = boston_data.iloc[:, :-1]
y = boston_data['Price']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=2
)

mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

X_train = X_train.fillna(X_train.mean())
X_test = X_test.fillna(X_test.mean())

from sklearn.linear_model import LinearRegression
regressor = LinearRegression()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

from sklearn.metrics import mean_squared_error, r2_score

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("RMSE :", rmse)
r2 = r2_score(y_test, y_pred)
print("R2 Score :", r2)

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

from keras.models import Sequential
from keras.layers import Dense

model = Sequential()
model.add(Dense(128, activation='relu', input_dim=13))

```

```
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(16, activation='relu'))
model.add(Dense(1))
```

```
model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=['mae']
)
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    validation_split=0.05
)
```

```
import plotly.graph_objects as go
fig = go.Figure()
fig.add_trace(
    go.Scatter(
        y=history.history['loss'],
        name='Train'
    )
)
fig.add_trace(
    go.Scatter(
        y=history.history['val_loss'],
        name='Valid'
    )
)
fig.update_layout(
    height=500,
    width=700,
    xaxis_title='Epoch',
    yaxis_title='Loss'
)
fig.show()
```

```
import plotly.graph_objects as go
fig = go.Figure()
fig.add_trace(
```

```

        go.Scatter(
            y=history.history['mae'],
            name='Train'
        )
    )
fig.add_trace(
    go.Scatter(
        y=history.history['val_mae'],
        name='Valid'
    )
)
fig.update_layout(
    height=500,
    width=700,
    xaxis_title='Epoch',
    yaxis_title='Mean Absolute Error'
)
fig.show()

y_pred = model.predict(X_test)
mse_nn, mae_nn = model.evaluate(X_test, y_test)
print("Mean squared error on test data :", mse_nn)
print("Mean absolute error on test data :", mae_nn)

from sklearn.metrics import mean_absolute_error
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)
mse_lr = mean_squared_error(y_test, y_pred_lr)
mae_lr = mean_absolute_error(y_test, y_pred_lr)
print("Mean squared error on test data :", mse_lr)
print("Mean absolute error on test data :", mae_lr)
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)
print(r2)

from sklearn.metrics import mean_squared_error
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(rmse)

import sklearn
new_data = sklearn.preprocessing.StandardScaler().fit_transform(
    [[0.1, 10.0, 5.0, 0, 0.4, 6.0, 50, 6.0, 1, 400, 20, 300, 10]]

```

```
)  
prediction = model.predict(new_data)  
print("Predicted house price:", prediction)
```